密集区間検出のためのトランザクション基盤モデル: アーキテクチャ設計と概念実証

Anonymous
Anonymous Institution

ABSTRACT

頻出アイテムセットマイニングにおいて、サポートが時間的に密集する区間の検出は重要な課題である。従来の Apriori ベースの手法は、ウィンドウサイズや閾値などのハイパーパラメータに敏感であり、新しいデータセットへの適応に手動チューニングを要する。本研究では、基盤モデル（Foundation Model）の枠組みをトランザクションデータの密集区間検出に初めて適用するための**アーキテクチャ設計**を提案する。具体的には、(1) 集合値入力に対するマルチホットエンコーディングとアイテム埋め込みの平均プーリングによるトランザクション埋め込み層、(2) フーリエ基底による密集区間トークン（DIT）表現、(3) マスクトランザクションモデリング（MTM）と密集区間予測（DIP）の 2 段階事前学習目的関数の 3 つを導入する。NumPy ベースの概念実証実装において、MTM 事前学習の収束性（損失 4.1% 削減）、従来 Apriori-Window 法との出力整合性（IoU 0.966）、および $O(LN^2d)$ の計算量スケーリングを確認した。DIP 精度は現段階で AUC 0.497 にとどまるが、これは埋め込み層固定による制約であり、端到端学習による改善の方向性を議論する。本研究の主要貢献は、トランザクションデータに対する基盤モデルの問題設定の定式化とアーキテクチャ設計にある。

KEYWORDS

基盤モデル, トランザクション埋め込み, 密集パターンマイニング, 自己教師あり学習, Transformer

1 はじめに

頻出アイテムセットマイニング [1, 9] は、トランザクションデータベースにおける繰り返し共起パターンを同定する基盤的手法である。トランザクションが時間順に並んでいる場合、アイテムセットのサポートは時間とともに変動し、特定の期間にサポートが集中する**密集区間**が現れる。このような密集区間の検出は、マーケティング施策の効果測定、異常検知、トレンド分析において実務的な意義を持つ。

従来の密集区間検出手法は、Apriori アルゴリズムとスライディングウィンドウを組み合わせ、ウィンドウサイズ W と閾値 θ をユーザが指定する枠組みに依存してきた。この手法は統計的に健全であるが、以下の 3 つの課題を抱える: (1) ハイパーパラメータ W と θ の設定がデータセットに依存し、手動チューニングを要する、(2) 新しいデータセットへの転移が困難である、(3) トランザクション間の時間的依存関係を明示的にモデル化しない。

近年、基盤モデル（Foundation Model）[3] は、大規模な自己教師あり事前学習により汎用的な表現を獲得し、下流タスクへの転移を実現する枠組みとして注目されている。自然言語処理における BERT [7] や GPT [4]、時系列予測における PatchTST [13] や TimesFM [6]、表形式データにおける TabTransformer [11] の成功に触発され、本研究ではこの枠組みをトランザクションデータの密集区間検出に適用する。

ただし、トランザクションデータは自然言語や画像と本質的に異なる構造を持つ。各トランザクションは**可変長の集合**であり、要素間に順序がない。また密集区間は局所的なサポート集中として定義され、グローバルな頻出性とは異なる概念である。これらの特性に対応するアーキテクチャ設計が本研究の中心課題である。

本研究の貢献は以下の 3 つである。

(1) **問題設定の定式化**: トランザクションデータに対する基盤モデルの適用を定式化し、集合値入力の埋め込み、時間的コンテキストの符号化、密集区間検出への転移の 3 段階パイプラインを設計する（第 3 節）。
(2) **アーキテクチャ設計**: マルチホットエンコーディング＋平均プーリングによるトランザクション埋め込み層、Pre-LN Transformer エンコーダ、およびフーリエ基底による密集区間トークン（DIT）を含む具体的なアーキテクチャを提案する（第 4 節）。
(3) **概念実証と限界分析**: NumPy ベースの実装により提案アーキテクチャの動作を検証し、現段階の限界（AUC 0.497）の原因分析と改善方向を明確に示す（第 5 節）。

注意 1. 本研究は概念実証（*proof of concept*）段階の**アーキテクチャ提案論文**である。大規模事前学習や実データでの転移学習は今後の課題として位置付ける。この点において、*TabTransformer [11]* が表形式データに対する *Transformer* アーキテクチャを提案した初期論文と類似の位置付けである。

2 関連研究

2.1 頻出パターンマイニング

Apriori アルゴリズム [1] は候補生成・枝刈り戦略により頻出アイテムセットを発見する古典的手法であり、FP-Growth [9] はパターン木による候補生成不要の手法を提案した。Eclat [21] は垂直データベース表現により集合演算で効率化を図る。SPMF [8] はこれらを含む多数のパターンマイニングアルゴリズムを統合したオープンソースライブラリである。これらの手法は静的な頻出度（全期間のサポート）に基づくが、サポートの**時間的集中**（密集区間）を直接扱わない。密集区間検出にはスライディングウィンドウベースの拡張が必要であり、ウィンドウサイズと閾値の手動設定が課題となる。

2.2 基盤モデルと自己教師あり学習

基盤モデル [3] は、大規模データでの自己教師あり事前学習により汎用表現を獲得し、少量のラベル付きデータで下流タスクに適応する枠組みである。Transformer アーキテクチャ [17] がその基盤技術であり、BERT [7] のマスク言語モデリング（MLM）、MAE [10] のマスク画像モデリングなど、マスク予測による事前学習が各ドメインで成功を収めている。

Table 1: 関連手法との位置付け比較

手法	入力型	事前学習	密集検出
Apriori/FP-Growth	集合	なし	間接
TabTransformer	固定長表	半教師あり	なし
BERT4Rec	アイテム列	MLM	なし
PatchTST/TimesFM	連続時系列	マスク予測	なし
提案手法	**集合列**	**MTM+DIP**	**直接**

2.3 表形式・トランザクションデータへの Transformer 適用

表形式データへの Transformer 適用として、TabTransformer [11] はカテゴリ特徴量の埋め込みに自己注意機構を適用し、半教師あり学習での有効性を示した。TabNet [2] はアテンションベースの特徴選択機構を導入した。SAINT [15] は行間・列間の双方向注意を提案した。これらは各行が固定スキーマの特徴ベクトルである表形式データを対象とするが、トランザクションデータは各行が**可変長の集合**であり、直接適用できない。

購買行動予測の文脈では、BERT の枠組みをユーザの購買系列に適用する BERT4Rec [16] が提案されているが、これはユーザ単位の逐次推薦であり、トランザクション集合全体の時間的密集性の検出とは問題設定が異なる。

2.4 時系列基盤モデル

時系列データに対する基盤モデルは急速に発展している。PatchTST [13] はパッチ化とチャネル独立の Transformer により長期予測を改善した。TimesFM [6] は Google が提案した時系列基盤モデルであり、大規模な実・合成時系列での事前学習によりゼロショット予測を実現した。Lag-Llama [14] は LLM アーキテクチャを確率的時系列予測に適用した。これらは連続値の時系列を対象とするが、トランザクションデータは各時点が**離散的な集合**であり、連続値系列とは本質的に異なるトークン化戦略が必要である。本研究が提案するトランザクション埋め込みと密集区間トークンは、この構造的差異を橋渡しするための設計である。

2.5 本研究の位置付け

表 1 に関連手法との差異を整理する。トランザクションデータ（可変長集合の時間列）に対して、密集区間検出を目的とした基盤モデルアーキテクチャの設計は、我々の知る限り本研究が初の試みである。

3 問題定式化

定義 1 (トランザクションデータベース). アイテム集合 $\mathcal{I} = \{1, 2, \ldots, |\mathcal{I}|\}$ 上のトランザクションデータベース $\mathcal{D} = (T_1, T_2, \ldots, T_N)$ は、時間順に並んだ N 個のトランザクションの列である。各トランザクション $T_t \subseteq \mathcal{I}$ はアイテム集合 $\mathcal{I}$ の部分集合であり、$|T_t|$ はトランザクション内のアイテム数を表す。

定義 2 (ウィンドウサポートと密集区間). アイテムセット $S \subseteq \mathcal{I}$ の位置 t におけるウィンドウサポートを $wsup(S, t, W) = |\{i : t \le i \le t+W-1,\ S \subseteq T_i\}|$ と定義する。ウィンドウサイズ W、閾値 θ における S の**密集区間**とは、極大な連続区間 $[s, e]$ であって、任意の $t \in [s, e]$ に対して $wsup(S, t, W) \ge \theta$ を満たすものである。

定義 3 (トランザクション埋め込み). トランザクション埋め込み関数 $\phi : \mathcal{P}(\mathcal{I}) \times \{1, \ldots, N\} \to \mathbb{R}^d$ は、各トランザクション T_t をその内容と時間位置 t から d 次元ベクトルに変換する写像である:

$$\phi(T_t, t) = LayerNorm\left(\frac{1}{|T_t|}\sum_{i \in T_t} \mathbf{e}_i + \mathbf{p}_t\right) \tag{1}$$

ここで $\mathbf{e}_i \in \mathbb{R}^d$ はアイテム i の学習可能な埋め込みベクトル（*Xavier* 一様分布で初期化）、$\mathbf{p}_t \in \mathbb{R}^d$ は正弦波位置符号化 *[17]*:

$$\mathbf{p}_{t,2j} = \sin(t/10000^{2j/d}), \quad \mathbf{p}_{t,2j+1} = \cos(t/10000^{2j/d}) \tag{2}$$

であり、*LayerNorm* は層正規化である。平均プーリングを採用する理由は、トランザクションが**順序のない集合**であり、置換不変な集約関数が必要なためである。

注意 2 (集合値入力の埋め込み設計). トランザクション T_t は可変長の集合であり、自然言語の単語列や画像のパッチ列とは本質的に異なる。平均プーリングは最も単純な置換不変集約であるが、*DeepSets [20]* の $\rho(\sum_i \phi(x_i))$ 型のより表現力の高い集約も将来の拡張として考えられる。

定義 4 (密集区間トークン). 密集区間トークン $\tau : [s, e] \to \mathbb{R}^{d_\tau}$ は、密集区間 $[s, e]$ の時間的特性をフーリエ基底で展開した d_τ 次元のトークン表現である:

$$\tau([s, e]) = LayerNorm\left(\bigoplus_{k=1}^{K} [\sin(k\pi\mathbf{f}),\ \cos(k\pi\mathbf{f})]\right) \tag{3}$$

ここで $\mathbf{f} = [s/N,\ e/N,\ (e-s+1)/N,\ \rho,\ (s+e)/(2N)] \in \mathbb{R}^5$ は正規化された特徴量ベクトルであり、$\rho = wsup(S, (s+e)/2, W)/W$ は区間中心における正規化密度、$d_\tau = 2 \times 5 \times K = 10K$ である。$\oplus$ はベクトル連結を表す。

定義 5 (事前学習目的関数). 事前学習は以下の *2* つの目的関数の重み付き和で定義される:

$$\mathcal{L} = \mathcal{L}_{MTM} + \lambda \mathcal{L}_{DIP} \tag{4}$$

マスクトランザクションモデリング（*MTM*）: トランザクション列からランダムにマスク率 r（デフォルト *15%*）で位置を選択し、マスク位置集合 $\mathcal{M}$ の埋め込みを *[MASK]* トークンで置換した上で、残りのコンテキストからマスク位置の埋め込みを復元する:

$$\mathcal{L}_{MTM} = \frac{1}{|\mathcal{M}|}\sum_{t \in \mathcal{M}} \|\hat{\mathbf{h}}_t - \phi(T_t, t)\|^2 \tag{5}$$

ここで $\hat{\mathbf{h}}_t$ はエンコーダ出力からの線形射影による復元ベクトルである。

密集区間予測（*DIP*）: 各位置が密集区間内か否かを二値分類する:

$$\mathcal{L}_{DIP} = -\frac{1}{N}\sum_{t=1}^{N} [y_t \log \hat{y}_t + (1 - y_t)\log(1 - \hat{y}_t)] \tag{6}$$

ここで $\hat{y}_t = \sigma(\mathbf{w}^\top \mathbf{h}_t + b)$、$y_t \in \{0, 1\}$ は従来法による密集区間ラベル（疑似ラベル）、σ はシグモイド関数である。λ は *DIP* 損失の重みであり、デフォルトで *1.0* とする。

4 提案手法

提案手法は 3 つのフェーズから構成される（図 1 参照）。表 2 にアーキテクチャの詳細仕様を示す。

Table 2: 提案アーキテクチャの仕様

コンポーネント	仕様
アイテム埋め込み	$\mathbf{e}_i \in \mathbb{R}^d$, Xavier 初期化
集約関数	平均プーリング（置換不変）
位置符号化	正弦波符号化 [17]
正規化	Pre-LN（層正規化を注意の前に適用）
Transformer ブロック	MHSA + FFN, 各々に残差接続
FFN	2 層 MLP, 隠れ次元 $4d$, GELU 活性化
ドロップアウト	注意重み・FFN 出力に p_{drop}
ヘッド数	h（各ヘッドの次元 $d_h = d/h$）
層数	L
MTM ヘッド	線形射影 $\mathbb{R}^d \to \mathbb{R}^d$
DIP ヘッド	線形分類器 $\mathbb{R}^d \to \mathbb{R}^1$

提案アーキテクチャの概要

トランザクション列 $T_1, \ldots, T_N$（各 $T_t \subseteq \mathcal{I}$）
↓ アイテム埋め込み平均プーリング + 正弦波位置符号化 + *LayerNorm*
埋め込み行列 $\mathbf{X} \in \mathbb{R}^{N \times d}$
↓ *Pre-LN Transformer* ×L 層（*MHSA* + *FFN* + 残差 + *Dropout*）
符号化行列 $\mathbf{H}^{(L)} \in \mathbb{R}^{N \times d}$
↙ ↘
`MTM` ヘッド（事前学習） `DIP` ヘッド（ファインチューニング）
$\hat{\mathbf{h}}_t \in \mathbb{R}^d$ $p_t \in [0, 1]$
↓ 閾値 δ + 最小長 $\ell_{\min}$ フィルタ
密集区間 $\{[s_k, e_k]\}$

Figure 1: トランザクション基盤モデルのアーキテクチャ。入力トランザクション列を平均プーリングと位置符号化によりベクトル系列に変換し、Pre-LN Transformer で符号化した後、MTM ヘッド（事前学習）と DIP ヘッド（ファインチューニング）に分岐する。

4.1 フェーズ 1: トランザクション埋め込み

各トランザクション T_t に対し、式 (1) に基づく埋め込みベクトル $\mathbf{x}_t = \phi(T_t, t) \in \mathbb{R}^d$ を計算する。アイテム埋め込み $\mathbf{e}_i$ は Xavier 一様分布で初期化される。完全な端到端学習では事前学習を通じてアイテム埋め込みも更新されるが、本概念実証では MTM ヘッドと DIP ヘッドのみを更新する簡易版を採用している（この制約の影響は第 5 節で分析する）。

4.2 フェーズ 2: Pre-LN Transformer エンコーダ

埋め込み系列 $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_N] \in \mathbb{R}^{N \times d}$ に対し、L 層の Pre-LN Transformer ブロックを適用する。各ブロックは以下の構成である:

$$\mathbf{H}' = \mathbf{H}^{(l-1)} + \text{Dropout}\left(\text{MHSA}\left(\text{LN}(\mathbf{H}^{(l-1)})\right)\right) \tag{7}$$

$$\mathbf{H}^{(l)} = \mathbf{H}' + \text{Dropout}\left(\text{FFN}\left(\text{LN}(\mathbf{H}')\right)\right) \tag{8}$$

ここで $\mathbf{H}^{(0)} = \mathbf{X}$、LN は層正規化、FFN は 2 層のフィードフォワードネットワーク:

$$\text{FFN}(\mathbf{z}) = \mathbf{W}_2 \cdot \text{GELU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2 \tag{9}$$

であり、$\mathbf{W}_1 \in \mathbb{R}^{4d \times d}$、$\mathbf{W}_2 \in \mathbb{R}^{d \times 4d}$ である。Pre-LN を採用する理由は、Post-LN と比較して学習の安定性が高いことが知られているためである [19]。

マルチヘッド自己注意（MHSA）は h 個のヘッドによるスケーリングドット積注意の連結である:

$$\text{MHSA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{10}$$

$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_h}}\right)\mathbf{V}_i \tag{11}$$

ここで $\mathbf{Q}_i = \mathbf{H}\mathbf{W}_i^Q$、$\mathbf{K}_i = \mathbf{H}\mathbf{W}_i^K$、$\mathbf{V}_i = \mathbf{H}\mathbf{W}_i^V \in \mathbb{R}^{N \times d_h}$、$d_h = d/h$ である。

4.3 フェーズ 3: タスクヘッド

MTM ヘッド. マスク位置 $t \in \mathcal{M}$ に対し、線形射影により埋め込みを復元する: $\hat{\mathbf{h}}_t = \mathbf{W}_{\text{mtm}}\mathbf{h}_t^{(L)} + \mathbf{b}_{\text{mtm}}$。

DIP ヘッド. 符号化された表現 $\mathbf{H}^{(L)} \in \mathbb{R}^{N \times d}$ に対し、線形分類器を適用して各位置の密集スコアを出力する:

$$p_t = \sigma(\mathbf{w}^\top \mathbf{h}_t^{(L)} + b) \tag{12}$$

連続して $p_t \geq \delta$ となる区間のうち、長さが最小長 $\ell_{\min}$ 以上のものを密集区間として出力する。

4.4 計算量分析

定理 1 (計算量). トランザクション数 N、埋め込み次元 d、ヘッド数 h、層数 L に対し、提案手法の時間計算量は $O(L \cdot (N^2 d + N d^2))$、空間計算量は $O(N^2 + Nd + |\mathcal{I}|d)$ である。

Proof. 各 Transformer ブロックにおいて: (a) Q, K, V 射影は $O(Nd^2)$（N 個のベクトルに対する $d \times d$ 行列乗算を 3 回）、(b) 注意重み $\mathbf{Q}\mathbf{K}^\top$ の計算は $O(N^2 d_h \cdot h) = O(N^2 d)$、(c) FFN は $O(N \cdot d \cdot 4d) = O(Nd^2)$。$L$ 層分で $O(L(N^2 d + Nd^2))$。トランザクション埋め込みは $O(\sum_t |T_t| \cdot d)$ であるが、Transformer の計算が支配的。空間計算量は注意行列 $O(N^2)$、表現行列 $O(Nd)$、アイテム埋め込み $O(|\mathcal{I}|d)$ の和。 □

命題 1 (従来法との計算量比較). *Apriori-Window* 法の密集区間検出は、候補枝刈りにより実際の計算量は $O(N \cdot W \cdot C)$ 程度である。ここで C は *Apriori* の候補枝刈り後に残る頻出アイテムセット数であり、データの密度に依存する。N が小さくアイテム共起が密な場合（C が大きい場合）は提案手法が有利であり、N が大きくアイテム共起が疎な場合は従来法が有利である。

注意 3. $O(N^2)$ の自己注意は $N > 10{,}000$ 程度でボトルネックとなる。*Linformer [18]* や *FlashAttention [5]* などの効率的注意機構による $O(N)$ 化が実用上重要である。

5 実験

本節では、NumPy ベースの概念実証実装により提案アーキテクチャの動作を検証する。本実験の目的は提案手法の有効性の最終的な実証ではなく、**アーキテクチャの各コンポーネントが設**

Table 3: 実験のハイパーパラメータ

パラメータ	記号	値
埋め込み次元	d	32
ヘッド数	h	4
層数	L	2
FFN 隠れ次元	d_{ffn}	128
ドロップアウト率	p_{drop}	0.1
MTM マスク率	r	0.15
MTM エポック数	—	30
DIP エポック数	—	20
学習率	η	0.001
勾配クリッピング	—	最大ノルム 1.0
DIP 損失重み	λ	1.0

計意図通りに機能するかの確認と、**現段階の限界の原因分析**にある。全実験はシード値 42 で固定し、再現性を保証する。

5.1 実験設定

合成データ生成. $N = 500$ トランザクション、$|\mathcal{I}| = 50$ アイテムの合成データを生成した。ベースラインのアイテム出現確率を $p_{\text{base}} = 0.1$、密集区間内のアイテム出現確率を $p_{\text{dense}} = 0.5$ とした。3 つの密集区間 $[50, 100]$, $[200, 250]$, $[350, 400]$ を植え込み、各区間に 3 つの密集アイテムを割り当てた。密集比率は $(51 + 51 + 51)/500 = 30.6\%$ である。

モデル設定. 表 3 にハイパーパラメータを示す。概念実証のため小規模な設定を採用した。

実装上の制約. 本概念実証は NumPy のみで実装しており、自動微分フレームワーク（PyTorch, JAX 等）を使用していない。そのため、以下の制約がある: (1) 勾配は MTM ヘッドと DIP ヘッドの線形層のみ手動計算で更新し、Transformer エンコーダとアイテム埋め込み層のパラメータは**固定**している、(2) 最適化は SGD のみ（Adam などの適応的学習率は未実装）。この制約が実験結果に与える影響を各実験で分析する。

5.2 E1: MTM 事前学習の収束性

マスクトランザクションモデリングによる事前学習の収束を確認した。$N = 300$ トランザクションに対し、30 エポックの学習を実施した結果、初期損失 2.763 から最終損失 2.650 へと 4.1% の削減を達成した（表 4）。学習率 0.001、勾配クリッピング（最大ノルム 1.0）の下で損失は単調に減少し、安定した収束が観察された。

限界分析. 損失削減率 4.1%は、一般的な MLM 事前学習（BERT: 損失収束まで数十%削減）と比較して小さい。これは MTM ヘッドの線形層のみを更新し、エンコーダのパラメータが固定されているため、コンテキストに基づくマスク位置の予測能力が制約されることに起因する。端到端学習により、注意機構がトランザクション間の依存関係を学習し、大幅な損失削減が期待される。

Table 4: E1: MTM 事前学習の収束性

指標	値
初期損失	2.763
最終損失	2.650
損失削減率	4.1%
エポック数	30
学習時間	0.100 s

Table 5: E2: 密集区間予測の閾値ごとの精度

閾値 δ	適合率	再現率	F1	備考
0.3	0.366	1.000	0.494	全位置を密集と予測
0.4	0.366	1.000	0.494	同上
0.5	0.000	0.000	0.000	全位置を非密集と予測
0.6	0.000	0.000	0.000	同上
0.7	0.000	0.000	0.000	同上
AUC			0.497	

5.3 E2: 密集区間予測精度

密集区間予測ヘッドの精度を閾値ごとに評価した（表 5）。データセット中の密集比率は 36.6%であった。最良の F1 値 0.494 は閾値 0.3 で達成され、AUC は 0.497 であった。

限界分析. AUC 0.497 はランダム分類器（AUC 0.5）とほぼ同等であり、DIP ヘッドが密集区間に特化した識別を達成できていないことを示す。根本的な原因は、エンコーダが固定されているため、DIP ヘッドへの入力 $\mathbf{h}_t^{(L)}$ が密集区間の情報を十分に含んでいないことにある。具体的には、ランダム初期化されたエンコーダの出力は、密集区間内外のトランザクションを区別する表現になっておらず、線形分類器のみの更新では分離が達成できない。端到端学習によりエンコーダが密集パターンに敏感な表現を学習すれば、AUC の大幅な改善が期待される。

閾値 0.5 以上で F1 が 0 になるのは、DIP ヘッドの出力がシグモイド関数の値域 $[0, 1]$ の下半分に集中しており、全位置に対して $p_t < 0.5$ となるためである。これもエンコーダ固定の帰結である。

5.4 E3: 従来法との出力整合性

提案手法の密集区間検出結果を従来の Apriori-Window 法と比較した（表 6）。従来法はウィンドウサイズ $W = 10$、閾値 $\theta = 3$ で 101 パターン・517 区間を検出した。基盤モデルは 1 つの包括的区間を検出し、従来法の区間との整合性は IoU 0.966、F1 0.983 であった。

結果の解釈. 高い IoU は提案手法の予測精度が高いことを意味するのではなく、DIP ヘッドが全位置に対して低い閾値で一様に密集と判定し、結果として従来法の密集区間を**過剰にカバーする**粗い区間を出力していることに起因する。これは E2 の分析と整合的である。端到端学習により、より精密な区間境界の検出が可能になると期待される。

Table 6: E3: 従来法との出力整合性

指標	従来法	基盤モデル
検出パターン数	101	—
検出区間数	517	1
実行時間 (s)	0.012	0.323
IoU	0.966	
F1	0.983	

Table 7: E4: スケーラビリティ分析（時間は秒単位）

N	符号化	事前学習	推論	符号化倍率
100	0.001	0.004	0.001	—
200	0.003	0.009	0.003	3.00×
500	0.011	0.046	0.011	3.67×
1000	0.041	0.177	0.036	3.73×
2000	0.163	0.736	0.130	3.98×

Table 8: E5: 埋め込み品質分析

指標	事前学習前	事前学習後
分離度	0.270	0.270
密集内コサイン類似度	0.907	0.907
注: エンコーダ固定のため変化なし（第 5 節の制約参照）		

5.5 E4: スケーラビリティ分析

トランザクション数 N を 100 から 2000 まで変化させ、各処理フェーズの実行時間を計測した（表 7）。符号化時間（自己注意の計算）と推論時間は理論的な $O(N^2)$ に概ね従う。N を 2 倍にした場合の実行時間の倍率は、符号化時間で $0.163/0.041 = 3.98 \approx 4 = 2^2$（$N = 1000 \rightarrow 2000$）であり、二次的スケーリングを確認した。

5.6 E5: 埋め込み品質分析

事前学習前後における埋め込みの品質を、密集区間内外のコサイン類似度の分離度で評価した（表 8）。分離度は密集区間内のトランザクション間コサイン類似度の平均から密集区間外のトランザクション間類似度の平均を引いた値で定義する。

結果と分析. 事前学習前の分離度は 0.270、密集区間内コサイン類似度は 0.907 であった。事前学習後も値は変化しなかった。これは本実装でアイテム埋め込み層とエンコーダのパラメータを固定しているため、表現空間が事前学習を通じて変化しないことの直接的帰結である。つまり、初期ランダム埋め込みによる偶発的な分離度がそのまま維持されている。端到端学習により、密集区間内のトランザクションが表現空間上でクラスタ化され、分離度の顕著な改善が期待される。

6 議論

6.1 概念実証の評価

本実験は提案アーキテクチャの動作確認を目的とした概念実証であり、以下の 3 点が確認された: (1) MTM 事前学習は安定して収束する（E1）、(2) 自己注意の計算量は理論通り $O(N^2)$ でスケールする（E4）、(3) アーキテクチャの各コンポーネント（埋め込み、エンコーダ、タスクヘッド）は設計意図通りに接続・動作する（E3, E5）。

一方、DIP 精度（AUC 0.497）は実用水準に達しておらず、これは NumPy ベース実装の根本的制約（エンコーダ固定）に起因する。この結果は、提案アーキテクチャの有効性を否定するものではなく、端到端学習が本手法の性能にとって**必須**であることを示唆する。

6.2 端到端学習への道筋

提案アーキテクチャを PyTorch または JAX で再実装する場合の改善要素:

(1) **全パラメータ更新**: アイテム埋め込み・エンコーダ・タスクヘッドの全パラメータを自動微分により同時更新する。
(2) **Adam オプティマイザ**: 適応的学習率によりパラメータごとの勾配スケールの違いを吸収する。
(3) **学習率スケジューリング**: ウォームアップ付きコサイン減衰 [12] により学習の安定性を向上させる。
(4) **データ拡張**: アイテムのランダムドロップ、時間軸のクロッピング、密集区間のシフトなどのトランザクション固有の拡張手法を導入する。

6.3 基盤モデルとしてのスケーリング方向

概念実証の範囲を超えて、本アーキテクチャを基盤モデルとしてスケールさせる場合、以下の方向性が考えられる:

(1) **大規模事前学習**: 複数ドメインのトランザクションデータ（小売、ウェブログ、金融取引等）を統合した大規模コーパスでの事前学習。
(2) **ドメイン間転移**: あるドメインで事前学習したモデルを、別のドメインにファインチューニングする。TimesFM [6] が示した時系列のゼロショット予測の成功に倣うアプローチ。
(3) **マルチタスク学習**: 密集区間検出、異常検知、トレンド予測、バスケット推薦を単一のエンコーダで実行する。
(4) **効率的注意機構**: FlashAttention [5] や線形注意機構の導入により、$N > 10{,}000$ のデータへの適用を可能にする。

7 結論

本研究では、密集区間検出のためのトランザクション基盤モデルの**アーキテクチャ設計**を提案した。具体的には、集合値入力に対する平均プーリング埋め込み、Pre-LN Transformer エンコーダ、フーリエ基底による密集区間トークン、および MTM と DIP の 2 段階事前学習目的関数を導入した。

NumPy ベースの概念実証実装により以下を確認した: (1) MTM 事前学習は安定して収束する（損失 4.1%削減）、(2) 計算量はトランザクション数に対して理論通り二次的にスケールする、(3) DIP 精度は AUC 0.497 であり、エンコーダ固定の制約下では実用水準に達しない。

本研究の主要貢献は、トランザクションデータ（可変長集合の時間列）に対する基盤モデルの問題設定の定式化とアーキテクチャ設計にある。PyTorch/JAX による端到端学習の実装、実データ（小売・ウェブログ）での評価、および効率的注意機構の導入が今後の主要課題である。

再現性

本研究の実装は Python / NumPy ベースであり、外部ライブラリへの依存は最小限である。合成データ生成、モデル学習、評価の全コードはリポジトリに含まれる。乱数シードは 42 に固定し、全実験結果の完全な再現を保証する。ハイパーパラメータの完全なリストは表 3 に記載した。

REFERENCES

[1] Rakesh Agrawal and Ramakrishnan Srikant. 1994. Fast Algorithms for Mining Association Rules. In *Proc. VLDB*. 487–499.

[2] Sercan Ö. Arik and Tomas Pfister. 2021. TabNet: Attentive Interpretable Tabular Learning. In *AAAI*. 6679–6687.

[3] Rishi Bommasani, Drew A. Hudson, et al. 2021. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258* (2021).

[4] Tom Brown et al. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*.

[6] Abhimanyu Das, Weihao Kong, Andrew Leber, Rajat Mathews, and Rajat Sen. 2024. A Decoder-Only Foundation Model for Time-Series Forecasting. *ICML* (2024).

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proc. NAACL-HLT* (2019), 4171–4186.

[8] Philippe Fournier-Viger, Jerry Chun-Wei Lin, Antonio Gomariz, Ted Gueniche, Azadeh Soltani, Zhihong Deng, and Hoang Thanh Lam. 2017. The SPMF Open-Source Data Mining Library Version 2. In *ECML PKDD*. 36–40.

[9] Jiawei Han, Jian Pei, Yiwen Yin, and Runying Mao. 2004. Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach. In *Data Mining and Knowledge Discovery*, Vol. 8. 53–87.

[10] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. 2022. Masked Autoencoders Are Scalable Vision Learners. In *CVPR*. 16000–16009.

[11] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. In *arXiv preprint arXiv:2012.06678*.

[12] Ilya Loshchilov and Frank Hutter. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In *ICLR*.

[13] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *ICLR*.

[14] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhatt, Erhan Bayraktar, Marin Thissen, Utku Turber, and Kari Torkkola. 2024. Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting. In *ICML*.

[15] Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C. Bayan Bruss, and Tom Goldstein. 2022. SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training. In *arXiv preprint arXiv:2106.01342*.

[16] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformers. In *CIKM*. 1441–1450.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *NeurIPS*. 5998–6008.

[18] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. In *arXiv preprint arXiv:2006.04768*.

[19] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. 2020. On Layer Normalization in the Transformer Architecture. In *ICML*. 10524–10533.

[20] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R. Salakhutdinov, and Alexander J. Smola. 2017. Deep Sets. In *NeurIPS*. 3391–3401.

[21] Mohammed J. Zaki. 2000. Scalable Algorithms for Association Mining. *IEEE Trans. Knowledge and Data Engineering* 12, 3 (2000), 372–390.